Week 4 Level 2: Intermediate

While Loops and Game Loops

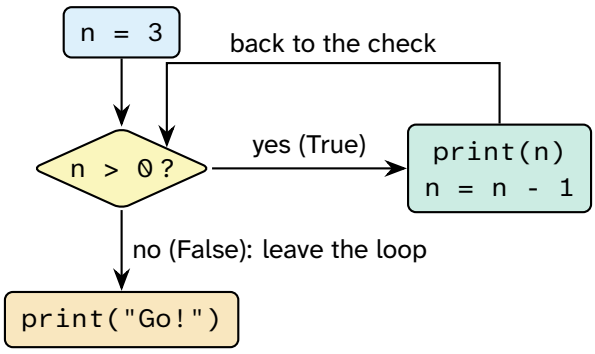
A while loop repeats until something changes, and every game is a loop.

What you will learn

- How a **while loop** keeps going until something changes.
- How to spot and stop an **infinite loop**.
- How to keep asking until the player types a good answer (**input validation**).
- How a **loop inside a loop** turns a list of strings into pixel art.

The big idea: repeat until something changes

A `for` loop knows how many times it will go round. A `while` loop does not: it checks a condition at the top, and goes round again *while* the condition is `True`.



This prints 3, 2, 1, Go!. The line `n = n - 1` is the **change**: without it, n stays 3, the check is always `True`, and the loop never ends. That is an **infinite loop**.

Tip: Stuck in a loop?

Press the red **Stop** button in Thonny's toolbar. Then ask two questions: **What is checked?** and **What changes inside the loop, so the check can become False?**

while True and break

Sometimes it is easier to loop "for ever" and jump out when you are done. `break` leaves the loop straight away. Inside a function, `return` leaves the loop *and* the function.

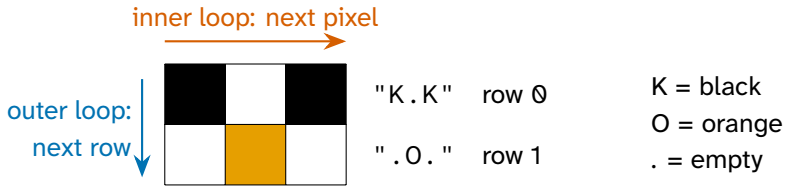
Keep asking until the answer is good

```
while True:
    answer = input("Type yes or no: ").lower()
    if answer == "yes" or answer == "no":
        break                          # good answer: leave the loop
    print("Please type yes or no.")
print("Thanks!")
```

Checking the player's answer like this is called **input validation**. Games do it all the time, so a typo never crashes them. `answer.isdigit()` is `True` only if the answer is made of the digits `0`–`9`: then `int(answer)` is safe.

A loop inside a loop: pixel art

A picture can be a list of strings: one string for each row, one letter for each pixel. The **outer loop** goes down the rows. The **inner loop** goes along one row, pixel by pixel.




```
for row in grid:            # outer loop: one row at a time
    for pixel in row:       # inner loop: one letter at a time
        print(pixel)
```

Exercise 4.1: Guess My Number

Folder: `exercises/week04/ex-guess-number`. Open `starter.py` and press **Run**.

1. Guess until you are right. Does the game stop? If not, press **Stop**. Why does it keep going?

---

2. TODO 1: add 1 to `tries` each time round. TODO 3: `break` when the result is `"Correct"`.
3. Now type `banana`. Crash! TODO 2: make `ask_number()` keep asking until the answer is a

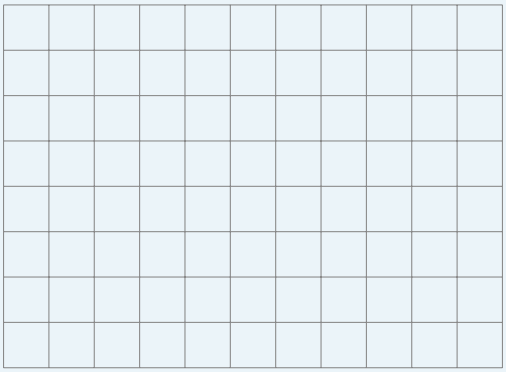
whole number from `low` to `high`.

4. Be the tester: try `banana`, `99`, `0` and `-3`. Does your game survive them all? ☐
5. Check your work with `check_my_work.py`.

Exercise 4.2: Pixel Art

Folder: `exercises/week04/ex-pixel-art`. Only the top row of the space invader appears.

1. TODO 1: wrap the drawing in an outer loop `for row in grid:`. Move down after each row: `y = y - size`.
2. TODO 2: finish `count_pixels()` with a loop inside a loop.
3. Design your own picture (8 rows of 11 letters fit nicely). Plan it here first:



Check your work with `check_my_work.py`.

Vocabulary box

while loop A loop that goes round again and again while its condition is True.

condition The check at the top of a while loop, like `n > 0`. It is True or False.

infinite loop A loop that never stops, because nothing changes the condition. Press Stop!

break Jumps out of a loop straight away.

input validation Checking the player's answer and asking again if it is not OK.

nested loop A loop inside another loop, like rows and pixels.

game loop The loop at the heart of a game: get input, update, show, repeat.

Reflection

When would you use a `while` loop instead of a `for` loop?

Name one way a game could be unfair to players. How would you make it fair?

Week 4 Level 2: Intermediate

While Loops and Game Loops

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Trace it first (paper, 4 minutes)

Before you code, follow `hits_needed(30, 7)` by hand. Each time round the loop, 7 is taken away from `health` and 1 is added to `hits`. Keep going until `health` is `0` or less.

round	health	hits
start	30	0
1	23	1
2		
3		
4		
5		

So `hits_needed(30, 7)` gives back: _____

★ Core (everyone): Slime Battle (Thonny, 10 minutes)

Open `exercises/week04/hw-slime-battle/starter.py`.

1. TODO 1: finish `hits_needed()` with a `while` loop: while `health` is more than `0`, take away `damage` and add 1 to `hits`.
2. TODO 2: make `ask_yes_no()` keep asking until the answer is `yes` or `no`.
3. Run it. Type `maybe` first. Does it ask again? ☐
4. Open `check_my_work.py` and press **Run**.

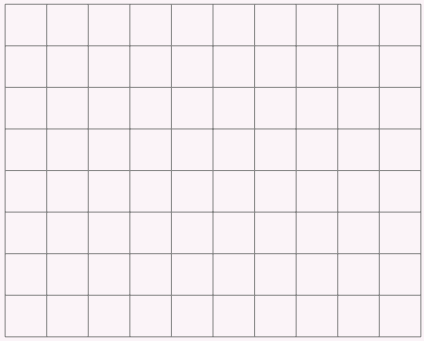
Stuck in a loop? Press the red **Stop** button and check that `health` really gets smaller inside the loop.

★★ Challenge (want more?): Count up to a target

Write a function `days_to_save(goal, per_day)` that uses a `while` loop to count how many days of saving `per_day` coins it takes to reach `goal` coins. For example `days_to_save(50, 12)` is 5. Add it to the bottom of your Slime Battle file (above `if __name__ == "__main__":`) and try it in the Shell.

Creative (make it yours): Pixel sprite design

Design a pixel sprite for a game: a slime, a pet, a spaceship or a potion. Color the squares and write the letter code for each row underneath (K, O, Y, B, S, G, R, P or . for empty). Next week you can type it into the Pixel Art program.



Self-check: tick each box when it is true

- ☐ I can explain what makes a `while` loop stop.
- ☐ I know what to do when a program is stuck in an infinite loop.
- ☐ My Slime Battle asks again when I type something that is not yes or no.
- ☐ All the checks in `check_my_work.py` pass (or I wrote down my question for next week).